

Syntax for Reflection

Document #: P3381R0 [\[Latest\]](#) [\[Status\]](#)
Date: 2024-09-16
Project: Programming Language C++
Audience: EWG
Reply-to: Wyatt Childers
<wcc@edg.com>
Peter Dimov
<pdimov@gmail.com>
Dan Katz
<dkatz85@bloomberg.net>
Barry Revzin
<barry.revzin@gmail.com>
Andrew Sutton
<andrew.n.sutton@gmail.com>
Faisal Vali
<faisalv@gmail.com>
Daveed Vandevoorde
<daveed@edg.com>

Contents

- [1 Introduction](#)
- [2 Single Character](#)
- [3 Multiple Characters](#)
- [4 Proposal](#)
- [5 Why Not A Keyword?](#)
- [6 References](#)

§ 1 Introduction

[\[P1240R2\]](#) originally proposed `^` as the reflection operator, which was also what [\[P2996R5\]](#) proposed and what both existing implementations (EDG and Clang) used. We've grown to really like the syntax: it's terse and stands out visually.

Unfortunately, it turns out that `^` is not a viable choice for reflection for C++26, due to the collision with the block extension used by Clang and commonly used in Objective-C++.

It was pointed out that the syntax:

```
type-id(^ident)();
```

is ambiguous. This can be parsed as both:

- a variable named `ident` holding a block returning `type-id` and taking no arguments, and
- a cast of `std::meta::info` (a reflection of `ident`) into a `type-id` and then a call of `operator()`.

This gets worse with the [P3294R1] usage of `^{ ... }` as a token sequence. This sequence is completely ambiguous:

```
auto A = ^{ f(); };
```

As such, the goal of this paper is to come up with new syntax for the reflection operator.

There are three approaches that we can go with this:

- a keyword,
- [a single character](#), or
- [multiple characters](#).

The original reflection design did use a keyword — `reflexpr(e)`. But that is far too verbose, which is why the design had changed to `^` to begin with. We would strongly prefer not to go back down that road.

That leaves either of the other choices.

§ 2 Single Character

There are not too many single character options available to us, presuming we want to stick with the characters that are easy to type and not just start perusing the available Unicode characters († is available after all). It also makes life much easier if we do not need to add new basic source characters (which a character like † would require).

Ignoring those characters that are already unary operators in C++ and the quotation marks, these are all the options available to us and what we think of them:

Token	Notes	Disposition
<code>#e</code>	The problem with <code>#e</code> is that precludes use in existing C macros. This code already has meaning: <pre>#define MACRO(x) do_something_with(#x)</pre> And that meaning is not taking the reflection of <code>x</code> and cannot change.	

\$e	Syntactically \$e is pretty nice and doesn't conflict with anything else in C++ since we only just added it to the basic character set. Unfortunately, compilers support the use of \$ in identifiers as an extension, so the simplest usage of \$T as the reflection of the type T is already ambiguous with the use an identifier that happens to start with a dollar sign.	✗
%e	We were initially fairly excited about the use of %e. Like ^e, this token is already used as a binary operator — but not an especially common one, so it seemed potentially viable as a reflection operator. Unfortunately, one way to use a reflection would be to pass it directly as the first template argument: <pre>C<%T> c;</pre> And it turns out that <% is a digraph, for {. This doesn't make it a complete non-starter, since this <i>is</i> something that can be worked around with use of parens or a space. But it's not great!	😞
,e	No.	✗
/e	The forward slash is the first character we come to that seems like a viable option. Here is some usage of it: <pre>constexpr auto r = /int; template for (constexpr auto e : enumerators_of(/E)) ; fn</R>(); [: /int :] [: /int:]</pre> Use of / also offers nice symmetry with the use of \ for interpolation in code injection (or perhaps arguably the wrong kind of symmetry, since the splice syntax we're not proposing to change). The downside of / is that it's pretty close to opening a comment. Now, we don't intend on //e to be valid syntax — reflecting an expression would require parentheses, so it would have to be /(/e). Ditto /*e is not valid, would have to be /(*e). Are those too close to comments?	✓

<code>:e</code>	Similar to the problems we ran into with <code>C<%T></code> not working because <code><%</code> is a digraph for <code>{</code> , <code>C<:T></code> would also not work because <code><:</code> is a digraph for <code>[</code> . If we're going to pick a token that has digraph problems, <code>%</code> is definitely the better option.	✗
<code>=e</code>	While <code>^</code> and <code>%</code> already exist as binary operators, they are fairly rare. <code>=</code> just seems way too common to be viable to overload to mean reflection	✗
<code>?e</code>	While <code>?</code> already exists in the language today, we do not believe that using <code>?e</code> would be ambiguous with the conditional operator. We could even come up with a story for why the question mark — we are asking what an entity is, and getting a reflection back to answer the question. It's just not our favorite, but it is potentially viable. <pre> constexpr auto r = ?int; template for (constexpr auto e : enumerators_of(?E)) ; fn<?R>(); [: ?int :] [:?int:] </pre> The downside of <code>?</code> is that this token tends to be strongly associated with predicates across programming languages. In C++ we just have the conditional operator, but in other languages we have null coalescing, optional chaining, and error propagation all spelled with <code>?</code> - plus for some languages it's convention to spell predicates as <code>v.empty?</code>, all of which has nothing to do with reflection. Pattern Matching ([P2688R1]) additionally proposes using <code>? pattern</code> as the optional pattern, which would conflict with the use of <code>?</code> as the reflection operator. So while it's technically available for use today, The optional pattern strikes us as a better use of unary <code>?</code> than reflection.	🙄
<code>@e</code>	<code>@</code> is already used as an extension in Objective-C. For many potential identifiers, there is already concrete meaning (<code>@property</code>, <code>@dynamic</code>, <code>@optional</code>, etc.). There is no way to escape this either, since <code>@(e)</code> is a boxed expression, and <code>@[e]</code> and <code>@{e}</code> are container literals.	✗
<code>\e</code>	Similar to <code>/e</code> , this is viable. But we prefer it as an interpolator (for which there is prior art), so we'd rather not use it here.	✗

	Additionally, this runs into issues with UCNs: <code>\u0</code> is parsed as a UCN, not a reflection of <code>u0</code> .	
<code>`e</code>	The third character recently added to the basic character set (after <code>\$</code> and <code>@</code>) is the backtick (or GRAVE ACCENT). The backtick has the advantage that it's pretty small, even smaller than <code>^</code>. But it has the disadvantage that backtick is used by Markdown everywhere inline code blocks, and not all Markdown implementations properly give you mechanisms to escape it. While not necessarily a show-stopper, we also just don't think it's good enough to reasonably pursue.	✗
<code> e</code>	The last available single token, this one is also viable, unambiguous, and not a part of a digraph: <pre>constexpr auto r = int; template for (constexpr auto e : enumerators_of(E)) ; fn< R>(); [: int :] [: int:]</pre> One potential issue with <code> </code> is when using multiple reflections in a single line, for instance: <pre>if (T == U)</pre> While not ambiguous to the compiler, those with math backgrounds might want to see <code> T == </code> as a magnitude of some sort.	✓

To summarize, there are only a few single tokens that we feel are completely viable:

- `/e`
- `|e`
- `?e` is viable today, but would compete with pattern matching
- `%e` if we're open to people struggling with digraphs

§ 3 Multiple Characters

Once we extend our search space to multiple characters, there are an infinite amount of possibilities to explorer — it's easy enough to come up with some sequence of tokens that is not a digraph and isn't ambiguous to parse.

For help with this investigation, Wyatt Childers put together a [simple utility](#) to test various alternatives for syntax. That utility allows you choose a prefix and/or suffix tokens and see what that looks like using various expected usage patterns.

Some choices that we have considered:

- `^^e` — While a single caret isn't viable, two carets would have no ambiguity with blocks. It's twice as long as the status quo, but requiring an additional character wouldn't be the end of the world. Two characters is short enough.
- `^[e]` — A different way to work around the block ambiguity is to throw more characters at it, in this case surrounding the operand with square brackets. On the one hand, this is more symmetric with splicing. On the other hand, it's a heavier syntax.
- `#{e}` or `$(e)` — One way to work around the identifier issue with `$` presented in the previous section is to use additional tokens that cannot be in identifiers. This would work fine for reflection, but doesn't have as nice a mirror for token sequences — would those use an extra set of braces? Additionally `$` seems more associated with interpolation than reflection in many languages, so simply seems like the opposite choice here. In contrast, `$(e)` might be an interesting choice for a splice operator — and could arguably have some symmetry with `^^e` with reflection (both having a double character).
- `/\e` — If we can't have a small caret, can we have a big caret? This actually has the same issue as just `\e` because of the UCN issue.

§ 4 Proposal

As a group, our preference is `^^e` (where `^^` is a new, single token).

It doesn't have any of the issues of the single-character solutions. Having the reflection operator be two characters as compared to only one isn't really a sufficiently large cost that we feel the need to reconsider it in favor of `|e`, `/e`, or `%e`. A two-character reflection operator is still overwhelmingly shorter than the 10-character operator that we started from (`reflexpr(e)`, if you count the parentheses), and that's good enough for us.

This has already been implemented in both EDG and Clang. Pending Evolution approval, we will simply update [\[P2996R5\]](#) to use the new syntax throughout — mostly just a search and replace, but with the extra addition in the wording of `^^` to the grammar of *operator-or-punctuator*.

§ 5 Why Not A Keyword?

The question that always comes up is: why some kind of punctuation mark (`^^` as proposed) instead of a keyword as originally proposed?

[P1240R0]	Proposed
<pre>constexpr auto r = reflexpr(S::m); auto m = s.unreflexpr(r);</pre>	<pre>constexpr auto r = ^^S::m; auto m = s.[:r:];</pre>

Whether that keyword is `reflexpr` or `reflectof` or `reflof` or `metaof`, we feel that a keyword would be the wrong choice for a reflection (or, especially, a splice) operator and are strongly opposed to that direction.

The primary reason for this is how heavy any keyword solution is compared to a punctuation solution, and how much that distracts from the intent of the code being presented.

We can start with a simple example of a typelist:

```
mp_list<signed char, short, int, long, long long>
```

Which corresponds to this as proposed, which still makes the types themselves stand out:

```
{^^signed char, ^^short, ^^int, ^^long, ^^long long}
```

But is much uglier and filled with tons of syntactic noise that doesn't contribute at all to readability if using a keyword:

```
{metaof(signed char), metaof(short), metaof(int), metaof(long),
 metaof(long long)}
```

The impulse to emphasize the reflection operation by making it stand out is, while understandable, fundamentally misguided. The code is never about taking a reflection; it's always about passing an entity to a function that then operates on it. The fact that we need to prefix the entity with `^^` is the price we're paying because entities aren't ordinary values so we need to apply an operator to turn them into ones. Not something to proudly write home about.

Or, in other words, we cannot 1 write the expression `f(int, float)` so we have to write `f(^^int, ^^float)`, which is the next best thing. Adding a ton of `metaof` is not a feature.

We can see this more clearly if we compare some language operators with their reflection equivalents. We have several operators in the language that take a type and produce a value of specific type — which makes them initially seem like precedent for how the reflection operator should behave. But:

Operation	Language Operator	Reflection
size	<code>sizeof(T)</code>	<code>size_of(^^T)</code>
alignment	<code>alignof(T)</code>	<code>align_of(^^T)</code>
is noexcept?	<code>noexcept(E)</code>	<code>is_noexcept(^^E)</code>

In all cases, the operation being performed is to get the size of the type (`sizeof` or `size_of`), the alignment of the type (`alignof` or `align_of`), or to check whether the expression is noexcept (`noexcept` or `is_noexcept`, noting that we don't have expression reflection yet — but when we do it'll look like this) and the operand is the type (T) or expression (E). Getting the reflection of the type/expression is *not* an important operation here. Making it stand out more does not have value. It would hide the actual operation.

The same is true for all operations that we might want to perform. Consider wanting to iterate over the enumerators of `Color`:

```
// reads as the enumerators of Color
enumerators_of(^Color)

// reads as the enumerators of the reflection of Color
enumerators_of(reflectof(Color))
```

Or to substituting `std::map` with `std::string` and `int`:

```
// reads as substitute map with string and int
substitute(^std::map, {^std::string, ^int})

// reads as substitute the reflection of map of with
// the reflection of string and the reflection of int
substitute(reflectof(std::map), {reflectof(std::string),
                                reflectof(int)})
```

The former reading more clearly expresses the intent. Keywords demand to be read, whereas sigils may be internally skipped over. Eliding the sigil from the internal dialogue lets the user put aside the fact that reflection is happening. They may read it as “enumerators of `Color`” or “substitute map with `string` and `int`.” Once the keyword-name enters the “internal token stream,” the user cannot hope to understand the meaning of the expression without learning the meaning of `reflectof` (or `metaof` or `reflof` or ...). That is exactly the opposite of novice-friendly.

A parallel can be made with templates. In `vector<int>` the template machinery is often “invisible”. the `<>` could be replaced by a keyword here as well. For instance, we could've had a keyword `substitute` and made forming a template something like `substitute(vector, int)` instead of `vector<int>`. However, we think most people would agree the punctuator puts the focus on the operation itself not the grammatical limitations of the language. Lots of novices engage in using templates long before writing templates, and while we've seen arguments in favor of a *different* punctuator for template arguments (such as `vector(int)` or D's `vector!(int)`, which avoid the `<` ambiguity), we've never seen an argument for a keyword here.

The other thing to point out is that reflection is not necessarily going to be a prominently user-facing facility. Certainly not a novice-facing one. Reflection opens the door to a large variety of libraries that can make otherwise very complex operations very easy — from even expert-unfriendly to novice-friendly. But the user-facing API of those libraries need not actually expose the reflection operator at all,

and most use-cases would not do so at all. Arguing for a reflection keyword to be novice-friendly thus doubly misses the point — not only is it not novice friendly, but novices may rarely even have to look at such code.

§ 6 References

- [P1240R0] Andrew Sutton, Faisal Vali, Daveed Vandevoorde. 2018-10-08. Scalable Reflection in C++. <https://wg21.link/p1240r0>
- [P1240R2] Daveed Vandevoorde, Wyatt Childers, Andrew Sutton, Faisal Vali. 2022-01-14. Scalable Reflection. <https://wg21.link/p1240r2>
- [P2688R1] Michael Park. 2024-02-15. Pattern Matching: `match` Expression. <https://wg21.link/p2688r1>
- [P2996R5] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-08-14. Reflection for C++26. <https://wg21.link/p2996r5>
- [P3294R1] Barry Revzin, Andrei Alexandrescu, Daveed Vandevoorde. 2024-07-16. Code Injection with Token Sequences. <https://wg21.link/p3294r1>
-

1. Consider the expression `f(X(int))`, where `X` is a type. This could be calling `f` with the function type `X(int)` or with a value of `X` constructed with `int`. How do you differentiate? ↩